

JS ES-6 Concepts

1. What is ES6 ?

- a. *ECMAScript 2015* (ES2015), also known as *ES6* , is a major update to JavaScript that introduced many new features and improvements to the language.
- b. It was the first big change since ES5 , which was released in 2009.
- c. Because ES2015 brought so many important changes, it's commonly referred to as ES6.

2. ES6 Concepts:

- a. **Here are the some major concepts which were released in ES6 in 2015:**
 - i. let and const
 - ii. Function with default parameters
 - iii. Arrow Function
 - iv. Rest Parameter and Spread Operator (2017)
 - v. Destructuring
 - vi. Template Literals (Backticks)
 - vii. Symbol
 - viii. Modules
 - ix. Promise
 - x. async and await (2017)
 - xi. Class

1. let and const:

- a. **let** and **const** are *block-scoped* variable declarations.
- b. The **let** variables are mutable i.e. their values can be changed. It works similar to the var keyword with some key differences like scoping which makes it a better option when compared to var
- c. The **const** is used to declare variables with a constant value, ensuring the value cannot be reassigned.

d. For Example →

```
let x = 10;  
x = 20; // ✓ allowed
```

```
const y = 30;  
// y = 40; ✗ error (cannot reassign)
```

2. Function with default parameters:

- a. Functions can have default values for parameters.
- b. Default parameters allow you to assign default values to function parameters if they are not provided during function calls.

c. For Example →

```
function greet(name = "Guest") {  
    return `Hello, ${name}`;  
}  
  
console.log(greet()); // Hello, Guest  
console.log(greet("John")); // Hello, Rohit
```

3. Arrow Function:

- a. Arrow functions is a shorter syntax for functions.
- b. They do not have their own this , so they inherit it from the surrounding context.
- c. For Example →

```
// normal function
function add(a, b) {
  return a + b;
}

// arrow function
const addArrow = (a, b) => a + b;

console.log(addArrow(5, 3)); // 8
```

4. Rest Parameter and Spread Operator (2017):

- a. Rest Parameter:
 - i. Collects all remaining parameters in an array.
 - ii. The rest parameter is used when defining a function to gather all remaining arguments into an array.
 - iii. It is used inside a **function definition**.
- iv. For Example →

```
function maxNumber(...max) {
  console.log(max); // array - (5) [1,2,3,4,5]
  return Math.max(...max);
}

console.log(maxNumber(1, 2, 3, 4, 5));
```

b. Spread Operator:

- i. The spread operator is used to *spread* or *unpack* the elements of an array (or object) into individual elements.
- ii. It is used inside a *function call*, or when copying arrays/objects.
- iii. **For Example** →

```
function maxNumber(...max) {  
  //Here, ...max => Rest Parameter  
  console.log(max);  
}  
  
let numbers = [1, 2, 3, 4, 5];  
  
maxNumber(...numbers);  
// Here ...numbers => Spread Operator
```

iv. Array Merging By Using Spread Operator:

```
const arr1 = [1, 2];  
const arr2 = [3, 4];  
const merged = [...arr1, ...arr2];  
console.log(merged); // [1,2,3,4]
```

v. Object Merging By Using Rest Parameter:

```
const obj1 = { a: 1 };  
const obj2 = { b: 2 };  
const obj3 = { ...obj1, ...obj2 };  
console.log(obj3); // {a:1, b:2}
```

5. Destructuring:

- a. **Destructuring** means breaking a big structure like an array or object into **smaller pieces (variables)** easily.
- b. Destructuring means to break down the complex structure into simpler parts.
- c. We can perform destructuring for arrays and objects for ease of use.
- d. It is a feature of ES6 which was introduced in 2015.

e. Array Destructuring:

- i. With the syntax of destructuring, we can extract the elements from arrays and store them inside fragments for ease of use.
- ii. **Syntax** →



```
let [var1, var2, varN] = array_ref_variable
```

iii. For Example →

```
// Array Destructuring
const arr = [10, 20];
const [first, second] = arr;
console.log(first, second); // 10 20
```

f. Rest Parameter in Array Destructuring:

- i. You can use the rest operator (**...**) to collect the remaining elements into an array.
- ii. **For Example** →

```
let numbers = [1, 2, 3, 4, 5];
```

```
// Array destructuring by using Rest  
Parameter  
let [x, y, ...restNum] = numbers;  
  
console.log(x, y); // 1 2  
console.log(restNum); // [3, 4, 5]
```

g. Object Destructuring:

- i. It is used to *extract the properties* of an object and bind them to variables for ease of use.
- ii. Extracts values from objects and assigns them to variables.
- iii. **Syntax** →

```
 let {var1, var2, varN} = obj_ref_variable
```

- iv. **NOTE** : *In object destructuring we have to pass the actual key names as variables.*
- v. **For Example** →

```
let person = { name: "John", id: 101,  
address: "New York" };
```

```
// Object Destructuring  
let { name, id, address } = person;  
  
console.log(name); // John  
console.log(id); // 101  
console.log(address); // New York
```

6. Template Literals (Backticks):

- a. Template literals allow for embedding expressions inside string literals.
- b. We have to use `` (backticks) for template literals.
- c. With the help of template literals we can perform *String Interpolation* (`${ }`).
- d. For Example →

```
const name1 = "John";  
console.log(`Hello, ${name1}! Welcome to ES6.`);  
// Hello, John! Welcome to ES6.
```

7. Symbol:

- a. *Symbol* is a primitive data type used to create unique identifiers for object properties.
- b. For Example →

```
const id = Symbol("id");  
const user = {  
  name: "John",  
  [id]: 101,  
};  
console.log(user); //{name: 'John', [Symbol(id)]: 101}
```

8. Modules:

- a. Modules will be discussed separately.

9. Promise:

- a. Promise will discuss in Asynchronous Javascript or in Fetching the API or url or data.
- b. We have a separate topic called Promises in JavaScript.

10. async and await (2017):

- a. async and await will be discussed in Promises.
- b. So in simple terms async and await keywords are used to handle the promises in Javascript.

11. Class:

- a. Class will be discussed in OOPS concept javascript.